

LW_HW3_Lokesh_Muvva_Coding

September 21, 2025

```
[1]: import pandas as pd
import numpy as np
import statsmodels.api as sm
import statsmodels.formula.api as smf
import statsmodels.stats.anova as anova
from statsmodels.stats.outliers_influence import summary_table
from scipy import stats
import matplotlib.pyplot as plt
```

```
[2]: print("="*60)
print("CODING QUESTION 1: Insurance Dataset Analysis")
print("="*60)
```

```
=====
CODING QUESTION 1: Insurance Dataset Analysis
=====
```

```
[3]: insurance_data = pd.read_csv('/Users/lokeshmuvva/Documents/regression_f25/data/
↳insurance.csv')
print(insurance_data.head())
```

	age	sex	bmi	children	smoker	region	charges
0	19	female	27.900	0	yes	southwest	16884.92400
1	18	male	33.770	1	no	southeast	1725.55230
2	28	male	33.000	3	no	southeast	4449.46200
3	33	male	22.705	0	no	northwest	21984.47061
4	32	male	28.880	0	no	northwest	3866.85520

```
[4]: X = insurance_data[['age', 'bmi', 'children']]
y = insurance_data['charges']

X = sm.add_constant(X)
```

```
[5]: # Fit the model
model = smf.ols('charges~age+bmi+children',data=insurance_data).fit()
```

```
[6]: print("Model: charges ~ age + bmi + children")
print("\nModel Summary:")
print(model.summary())
```

Model: charges ~ age + bmi + children

Model Summary:

OLS Regression Results						
=====						
Dep. Variable:	charges		R-squared:	0.120		
Model:	OLS		Adj. R-squared:	0.118		
Method:	Least Squares		F-statistic:	60.69		
Date:	Sun, 21 Sep 2025		Prob (F-statistic):	8.80e-37		
Time:	23:30:14		Log-Likelihood:	-14392.		
No. Observations:	1338		AIC:	2.879e+04		
Df Residuals:	1334		BIC:	2.881e+04		
Df Model:	3					
Covariance Type:	nonrobust					
=====						
	coef	std err	t	P> t	[0.025	0.975]

Intercept	-6916.2433	1757.480	-3.935	0.000	-1.04e+04	-3468.518
age	239.9945	22.289	10.767	0.000	196.269	283.720
bmi	332.0834	51.310	6.472	0.000	231.425	432.741
children	542.8647	258.241	2.102	0.036	36.261	1049.468
=====						
Omnibus:	325.395		Durbin-Watson:	2.012		
Prob(Omnibus):	0.000		Jarque-Bera (JB):	603.372		
Skew:	1.520		Prob(JB):	9.54e-132		
Kurtosis:	4.255		Cond. No.	290.		
=====						

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```
[7]: print("\n" + "="*50)
print("PART (a): Model Interpretation")
print("="*50)

# Extract key statistics
f_stat = model.fvalue
f_pvalue = model.f_pvalue
r_squared = model.rsquared
adj_r_squared = model.rsquared_adj

print("\nInterpretation:")
print(f"1. Global F-test: {f_stat:.4f}")
if f_pvalue < 0.05:
    print(f"    - P-value = {f_pvalue:.6f} < 0.05, so we reject H ")
    print("    - The model is statistically significant overall")
```

```

    print("    - At least one predictor has a non-zero coefficient and is a
    ↪significant predictor")
else:
    print(f"    - P-value = {f_pvalue:.6f}    0.05, so we fail to reject H")
    print("    - The model is not statistically significant overall")

print(f"\n2. R-squared = {r_squared:.4f}:")
print(f"    - The model explains {r_squared*100:.2f}% of the variance in
    ↪charges")

print(f"\n3. Adjusted R-squared = {adj_r_squared:.4f}:")
print(f"    - Adjusted for number of predictors: {adj_r_squared*100:.2f}%")
if adj_r_squared < r_squared:
    print("    - Lower than R2, indicating as a result of the change/difference
    ↪due to additional predictors")

print("\n4. Individual coefficient t-tests:")
for i, coef_name in enumerate(model.params.index):
    coef_val = model.params.iloc[i]
    p_val = model.pvalues.iloc[i]
    print(f"    - {coef_name}: coeff = {coef_val:.4f}, p-value = {p_val:.6f}")
    if p_val < 0.05:
        print(f"        → Significant at    = 0.05 level")
    else:
        print(f"        → Not significant at    = 0.05 level")

```

```

=====
PART (a): Model Interpretation
=====

```

Interpretation:

1. Global F-test: 60.6928
 - P-value = 0.000000 < 0.05, so we reject H
 - The model is statistically significant overall
 - At least one predictor has a non-zero coefficient and is a significant predictor
2. R-squared = 0.1201:
 - The model explains 12.01% of the variance in charges
3. Adjusted R-squared = 0.1181:
 - Adjusted for number of predictors: 11.81%
 - Lower than R², indicating as a result of the change/difference due to additional predictors
4. Individual coefficient t-tests:

- Intercept: coeff = -6916.2433, p-value = 0.000087
→ Significant at = 0.05 level
- age: coeff = 239.9945, p-value = 0.000000
→ Significant at = 0.05 level
- bmi: coeff = 332.0834, p-value = 0.000000
→ Significant at = 0.05 level
- children: coeff = 542.8647, p-value = 0.035726
→ Significant at = 0.05 level

```
[8]: print("\n" + "="*50)
print("PART (b): Sequential (Type I) Sum of Squares")
print("="*50)

# Type I (Sequential) ANOVA
anova_type1 = anova.anova_lm(model, typ=1)
print("Type I (Sequential) ANOVA:")
print(anova_type1)

print("\nInterpretation of 'bmi' in Type I ANOVA:")
print("- Reduced model: charges ~ age")
print("- Full model: charges ~ age + bmi")
print("- Sequential SS tests: Does adding 'bmi' after 'age' improve the model?")
print("- H: _bmi = 0 (given age is already in model)")
print("- H: _bmi = 0 (given age is already in model)")

bmi_p_type1 = anova_type1.loc['bmi', 'PR(>F)']
print(f"- P-value for bmi: {bmi_p_type1:.6f}")
if bmi_p_type1 < 0.05:
    print("- Conclusion: BMI significantly improves the model after accounting_
    ↳for age")
else:
    print("- Conclusion: BMI does not significantly improve the model after_
    ↳accounting for age")
```

```
=====
PART (b): Sequential (Type I) Sum of Squares
=====

Type I (Sequential) ANOVA:
```

	df	sum_sq	mean_sq	F	PR(>F)
age	1.0	1.753019e+10	1.753019e+10	135.546341	6.627851e-30
bmi	1.0	5.446449e+09	5.446449e+09	42.112843	1.211545e-10
children	1.0	5.715190e+08	5.715190e+08	4.419080	3.572625e-02
Residual	1334.0	1.725261e+11	1.293299e+08	NaN	NaN

Interpretation of 'bmi' in Type I ANOVA:

- Reduced model: charges ~ age
- Full model: charges ~ age + bmi

- Sequential SS tests: Does adding 'bmi' after 'age' improve the model?
- H: $\beta_{bmi} = 0$ (given age is already in model)
- H: $\beta_{bmi} = 0$ (given age is already in model)
- P-value for bmi: 0.000000
- Conclusion: BMI significantly improves the model after accounting for age

```
[9]: print("\n" + "="*50)
print("PART (c): Partial (Type II) Sum of Squares")
print("="*50)

# Type II (Partial) ANOVA
anova_type2 = anova.anova_lm(model, typ=2)
print("Type II (Partial) ANOVA:")
print(anova_type2)

print("\nInterpretation of 'bmi' in Type II ANOVA:")
print("- Reduced model: charges ~ age + children")
print("- Full model: charges ~ age + bmi + children")
print("- Partial SS tests: Is 'bmi' needed given that 'age' and 'children' are_
    ↪in the model?")
print("- H:  $\beta_{bmi} = 0$  (given age and children are in model)")
print("- H:  $\beta_{bmi} = 0$  (given age and children are in model)")

bmi_p_type2 = anova_type2.loc['bmi', 'PR(>F)']
print(f"- P-value for bmi: {bmi_p_type2:.6f}")
if bmi_p_type2 < 0.05:
    print("- Conclusion: BMI is significant after adjusting for age and_
    ↪children")
else:
    print("- Conclusion: BMI is not significant after adjusting for age and_
    ↪children")
```

```
=====
PART (c): Partial (Type II) Sum of Squares
=====
Type II (Partial) ANOVA:
```

	sum_sq	df	F	PR(>F)
age	1.499426e+10	1.0	115.938067	5.533923e-26
bmi	5.417280e+09	1.0	41.887301	1.354882e-10
children	5.715190e+08	1.0	4.419080	3.572625e-02
Residual	1.725261e+11	1334.0	NaN	NaN

```
Interpretation of 'bmi' in Type II ANOVA:
- Reduced model: charges ~ age + children
- Full model: charges ~ age + bmi + children
- Partial SS tests: Is 'bmi' needed given that 'age' and 'children' are in the
model?
```

- H: $\beta_{bmi} = 0$ (given age and children are in model)
- H: $\beta_{bmi} = 0$ (given age and children are in model)
- P-value for bmi: 0.000000
- Conclusion: BMI is significant after adjusting for age and children

```
[10]: print("\n" + "="*60)
print("CODING QUESTION 2: Property Dataset Analysis")
print("="*60)
```

```
=====
CODING QUESTION 2: Property Dataset Analysis
=====
```

```
[11]: # Load property dataset
# Load the data - first column is Y, then X1, X2, X3, X4
property_data = pd.read_csv('/Users/lokeshmuvva/Documents/regression_f25/data/
↳property.txt', sep='\s+', header=None)
property_data.columns = ['Y', 'X1', 'X2', 'X3', 'X4']
print(property_data.head())
```

	Y	X1	X2	X3	X4
0	13.5	1	5.02	0.14	123000
1	12.0	14	8.19	0.27	104079
2	10.5	16	3.00	0.00	39998
3	15.0	4	10.70	0.05	57112
4	14.0	11	8.97	0.07	60000

```
[12]: print("\n" + "="*50)
print("PART (a): Model Fitting and Summary")
print("="*50)

# Set up the regression model
X_prop = property_data[['X1', 'X2', 'X3', 'X4']]
y_prop = property_data['Y']

# Add intercept
X_prop = sm.add_constant(X_prop)

# Fit the model
property_model = sm.OLS(y_prop, X_prop).fit()

print("Property Model: Y ~ X1 + X2 + X3 + X4")
print("Where:")
print("Y = rental rates (thousands of dollars)")
print("X1 = age")
print("X2 = operating expenses (thousands of dollars)")
print("X3 = vacancy rate")
```

```
print("X4 = total square footage")
print("\nModel Summary:")
print(property_model.summary())
```

```
=====
```

PART (a): Model Fitting and Summary

```
=====
```

Property Model: $Y \sim X1 + X2 + X3 + X4$

Where:

Y = rental rates (thousands of dollars)

X1 = age

X2 = operating expenses (thousands of dollars)

X3 = vacancy rate

X4 = total square footage

Model Summary:

OLS Regression Results

```
=====
Dep. Variable:          Y      R-squared:          0.585
Model:                  OLS    Adj. R-squared:      0.563
Method:                 Least Squares    F-statistic:      26.76
Date:                  Sun, 21 Sep 2025    Prob (F-statistic):  7.27e-14
Time:                  23:30:14    Log-Likelihood:     -122.75
No. Observations:      81    AIC:              255.5
Df Residuals:          76    BIC:              267.5
Df Model:               4
Covariance Type:       nonrobust
=====
```

	coef	std err	t	P> t	[0.025	0.975]
const	12.2006	0.578	21.110	0.000	11.049	13.352
X1	-0.1420	0.021	-6.655	0.000	-0.185	-0.100
X2	0.2820	0.063	4.464	0.000	0.156	0.408
X3	0.6193	1.087	0.570	0.570	-1.545	2.784
X4	7.924e-06	1.38e-06	5.722	0.000	5.17e-06	1.07e-05

```
=====
Omnibus:                1.922    Durbin-Watson:          1.580
Prob(Omnibus):          0.383    Jarque-Bera (JB):        1.301
Skew:                   0.148    Prob(JB):                0.522
Kurtosis:               3.545    Cond. No.                1.74e+06
=====
```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 1.74e+06. This might indicate that there are strong multicollinearity or other numerical problems.

```
[13]: print("\n" + "="*50)
print("PART (b): Fitted Values, Residuals, and 2 Estimate")
print("="*50)

# Get fitted values and residuals
fitted_values = property_model.fittedvalues
residuals = property_model.resid

# Print first 6 cases
print("First 6 cases - Fitted Values and Residuals:")
results_df = pd.DataFrame({
    'Observed_Y': y_prop[:6],
    'Fitted_Y': fitted_values[:6],
    'Residuals': residuals[:6]
})
print(results_df)

# Estimate 2
n = len(y_prop)
p = X_prop.shape[1] # number of parameters including intercept
sse = np.sum(residuals**2)
sigma_squared = sse / (n - p)

print(f"\nEstimate of 2 based on the residuals:")
print(f"SSE = {sse:.4f}")
print(f"df = n - p = {n} - {p} = {n-p}")
print(f"2 = SSE/(n-p) = {sse:.4f}/{n-p} = {sigma_squared:.4f}")
print(f"2 = {np.sqrt(sigma_squared):.4f}")
```

```
=====
PART (b): Fitted Values, Residuals, and 2 Estimate
=====
```

First 6 cases - Fitted Values and Residuals:

	Observed_Y	Fitted_Y	Residuals
0	13.5	14.535672	-1.035672
1	12.0	13.513806	-1.513806
2	10.5	11.091053	-0.591053
3	15.0	15.133568	-0.133568
4	14.0	13.686716	0.313284
5	10.5	13.687185	-3.187185

Estimate of ² based on the residuals:

SSE = 98.2306

df = n - p = 81 - 5 = 76

² = SSE/(n-p) = 98.2306/76 = 1.2925

² = 1.1369


```
[14]: print("\n" + "="*50)
print("PART (c): Test    = 0 at    = 0.01")
print("="*50)

# Test H:    = 0 vs H:    0 at    = 0.01
beta2_coeff = property_model.params['X2']
beta2_se = property_model.bse['X2']
beta2_tstat = property_model.tvalues['X2']
beta2_pvalue = property_model.pvalues['X2']

print("Testing H:    = 0 vs H:    0")
print(f"Coefficient ^ = {beta2_coeff:.4f}")
print(f"Standard Error SE(^ ) = {beta2_se:.4f}")
print(f"t-statistic = {beta2_tstat:.4f}")
print(f"p-value = {beta2_pvalue:.6f}")
print(f"Significance level    = 0.01")

if beta2_pvalue < 0.01:
    print("Decision: Reject H (p-value < 0.01)")
    print("Conclusion:    is significantly different from 0 at    = 0.01 level")
else:
    print("Decision: Fail to reject H (p-value    0.01)")
    print("Conclusion:    is not significantly different from 0 at    = 0.01_
↪level")

print(f"\nInterpretation of ^ = {beta2_coeff:.4f}:")
print("This coefficient represents the expected change in rental rates, in_
↪thousands of dollars (Y)")
print("for each additional unit increase in operating expenses, in thousands of_
↪ dollars (X2), holding all other variables constant.")
```

```
=====
PART (c): Test    = 0 at    = 0.01
=====

Testing H:    = 0 vs H:    0
Coefficient ^ = 0.2820
Standard Error SE(^ ) = 0.0632
t-statistic = 4.4642
p-value = 0.000027
Significance level    = 0.01
Decision: Reject H (p-value < 0.01)
Conclusion:    is significantly different from 0 at    = 0.01 level

Interpretation of ^ = 0.2820:
This coefficient represents the expected change in rental rates, in thousands of
dollars (Y)
for each additional unit increase in operating expenses, in thousands of
```

dollars (X2), holding all other variables constant.

```
[15]: print("\n" + "="*50)
print("PART (d): Joint Test H:    =    = 0")
print("="*50)

# Test H:    =    = 0 vs H: at least one    0
# This requires fitting a reduced model without X1 and X2

# Reduced model: Y ~ X3 + X4
X_reduced = property_data[['X3', 'X4']]
X_reduced = sm.add_constant(X_reduced)
reduced_model = sm.OLS(y_prop, X_reduced).fit()

# Calculate F-statistic for the test
sse_reduced = np.sum(reduced_model.resid**2)
sse_full = np.sum(property_model.resid**2)
df_num = 2 # number of restrictions (testing    =    = 0)
df_den = n - p # degrees of freedom for full model

f_stat = ((sse_reduced - sse_full) / df_num) / (sse_full / df_den)
f_pvalue = 1 - stats.f.cdf(f_stat, df_num, df_den)

print("Joint test: H:    =    = 0 vs H: at least one    0")
print(f"SSE(reduced) = {sse_reduced:.4f}")
print(f"SSE(full) = {sse_full:.4f}")
print(f"F-statistic = {f_stat:.4f}")
print(f"df numerator = {df_num}")
print(f"df denominator = {df_den}")
print(f"p-value = {f_pvalue:.6f}")

alpha = 0.05 # typical significance level
if f_pvalue < alpha:
    print(f"Decision: Reject H (p-value < {alpha})")
    print("Conclusion: At least one of    or    is significantly different from_
↪0")
else:
    print(f"Decision: Fail to reject H (p-value    {alpha})")
    print("Conclusion: Neither    nor    is significantly different from 0")
```

```
=====
PART (d): Joint Test H:    =    = 0
=====
Joint test: H:    =    = 0 vs H: at least one    0
SSE(reduced) = 168.6523
SSE(full) = 98.2306
F-statistic = 27.2423
```

```

df numerator = 2
df denominator = 76
p-value = 0.000000
Decision: Reject H (p-value < 0.05)
Conclusion: At least one of    or    is significantly different from 0

```

```

[16]: print("\n" + "="*50)
print("PART (e): 90% Prediction Interval")
print("="*50)

# Prediction for X1=4, X2=10, X3=0.1, X4=80000
x_new = np.array([1, 4, 10, 0.1, 80000]) # include intercept
print("Property characteristics:")
print("X (age) = 4")
print("X (operating expenses) = 10 thousand dollars")
print("X (vacancy rate) = 0.1")
print("X (square footage) = 80,000")

# Point prediction
y_pred = np.dot(x_new, property_model.params)
print(f"\nPoint prediction:  $\hat{y}$  = {y_pred:.4f} thousand dollars")

# Prediction interval
# SE(prediction) =  $\sqrt{1 + x'(X'X)^{-1}x}$ 
XtX_inv = np.linalg.inv(X_prop.T @ X_prop)
prediction_var = sigma_squared * (1 + x_new.T @ XtX_inv @ x_new)
prediction_se = np.sqrt(prediction_var)

# 90% prediction interval
alpha = 0.10
t_critical = stats.t.ppf(1 - alpha/2, df=n-p)

lower_bound = y_pred - t_critical * prediction_se
upper_bound = y_pred + t_critical * prediction_se

print(f"\n90% Prediction Interval:")
print(f"SE(prediction) = {prediction_se:.4f}")
print(f"t-critical (=0.10, df={n-p}) = {t_critical:.4f}")
print(f"Lower bound = {lower_bound:.4f} thousand dollars")
print(f"Upper bound = {upper_bound:.4f} thousand dollars")
print(f"90% PI: [{lower_bound:.4f}, {upper_bound:.4f}] thousand dollars")

print("\nInterpretation:")
print("Based on the prediction interval, there is 90% confidence that the_
↪ rental rate for a property with the given")
print("characteristics will be between ${:.2f}k and ${:.2f}k.".
↪ format(lower_bound, upper_bound))

```

```
=====
PART (e): 90% Prediction Interval
=====
```

Property characteristics:

```
X (age) = 4
X (operating expenses) = 10 thousand dollars
X (vacancy rate) = 0.1
X (square footage) = 80,000
```

Point prediction: $\hat{y}$ = 15.1485 thousand dollars

90% Prediction Interval:

```
SE(prediction) = 1.1528
t-critical (=0.10, df=76) = 1.6652
Lower bound = 13.2289 thousand dollars
Upper bound = 17.0681 thousand dollars
90% PI: [13.2289, 17.0681] thousand dollars
```

Interpretation:

Based on the prediction interval, there is 90% confidence that the rental rate for a property with the given characteristics will be between \$13.23k and \$17.07k.

[17]: *# Data setup for Question #5 from Written Questions*

```
X = np.array([[1, 7, 33],
              [1, 4, 41],
              [1, 16, 7],
              [1, 3, 49],
              [1, 21, 5],
              [1, 8, 31]])

Y = np.array([42, 33, 75, 28, 91, 55])

# (a) Calculate  $\hat{\beta} = (X'X)^{-1}X'Y$ 
XtX = X.T @ X
XtY = X.T @ Y
XtX_inv = np.linalg.inv(XtX)
beta_hat = XtX_inv @ XtY

print(" ")
print("(a) Parameter estimates  $\hat{\beta}$ :")
print("\n $\hat{\beta}$  =", beta_hat)

# (b) Calculate residuals  $e = Y - \hat{Y}$ 
Y_hat = X @ beta_hat
e = Y - Y_hat
```

```

print("\n")
print("(b) Residuals  $e = Y - \hat{X}$ :")
print("e =", e)

# (c) Calculate hat matrix  $H = X(X'X)^{-1}X'$ 
H = X @ XtX_inv @ X.T

print(" ")
print("(c) Hat matrix  $H = X(X'X)^{-1}X'$ :")
print("H =")
print(H)

# (d) Calculate  $SSR = \hat{X}'Y - n\bar{y}^2$ 
n = len(Y)
y_bar = np.mean(Y)
SSR = beta_hat.T @ XtY - n * y_bar**2

print(" ")
print("(d) Sum of Squares Regression (SSR):")
print("SSR =  $\hat{X}'Y - n\bar{y}^2$  =", SSR)

# (e) Calculate  $SE(\hat{\beta}) = \sqrt{MSE \times \text{diagonal of } (X'X)^{-1}}$ 
SSE = e.T @ e
p = X.shape[1] # number of parameters
MSE = SSE / (n - p)
se_beta = np.sqrt(MSE * np.diag(XtX_inv))

print(" ")
print("(e) Standard errors  $SE(\hat{\beta})$ :")
print("SE( $\hat{\beta}$ ) =", se_beta)

# (f) Prediction  $\hat{Y}$  when  $X = 10$ ,  $X = 30$ 
x_h = np.array([1, 10, 30])
Y_h = x_h @ beta_hat

print(" ")
print("(f) Prediction  $\hat{Y}$  when  $X = 10$ ,  $X = 30$ :")
print(" $\hat{Y} = x' \hat{\beta}$  =", Y_h)

# (g) Standard error of prediction  $SE(\hat{Y})$ 
SE_Y_h = np.sqrt(MSE * x_h.T @ XtX_inv @ x_h)

print(" ")
print("(g) Standard error of prediction  $SE(\hat{Y})$ :")
print("SE( $\hat{Y}$ ) =  $\sqrt{MSE \times x' (X'X)^{-1} x}$  =", SE_Y_h)

```

(a) Parameter estimates $\hat{\beta}$:

$\hat{\beta} = [33.93210327 \quad 2.7847614 \quad -0.26441893]$

(b) Residuals $e = Y - \hat{Y}$:

$e = [-2.69960842 \quad -1.22997279 \quad -1.63735316 \quad -1.32985996 \quad -0.08999801 \quad 6.98679233]$

(c) Hat matrix $H = X(X'X)^{-1}X'$:

$H =$

```
[ [ 0.23143293  0.25167585  0.21178735  0.14886839 -0.05475543  0.21099091]
  [ 0.25167585  0.31240459  0.09437844  0.26627729 -0.14787283  0.22313666]
  [ 0.21178735  0.09437844  0.70442026 -0.31917435  0.10446672  0.20412159]
  [ 0.14886839  0.26627729 -0.31917435  0.61425632  0.14143492  0.14833743]
  [-0.05475543 -0.14787283  0.10446672  0.14143492  0.94039955  0.01632707]
  [ 0.21099091  0.22313666  0.20412159  0.14833743  0.01632707  0.19708635]]
```

(d) Sum of Squares Regression (SSR):

$SSR = \hat{\beta}'X'Y - n\bar{y}^2 = 3009.9264618034686$

(e) Standard errors $SE(\hat{\beta})$:

$SE(\hat{\beta}) = [26.7482922 \quad 1.28905638 \quad 0.51231607]$

(f) Prediction $\hat{Y}$ when $X = 10$, $X = 30$:

$\hat{Y} = x' \hat{\beta} = 53.84714939934844$

(g) Standard error of prediction $SE(\hat{Y})$:

$SE(\hat{Y}) = \sqrt{MSE \times x'(X'X)^{-1}x} = 2.329081299601817$